

Declaración de IA y otras Fuentes

Nicolas Casillas Larrañaga | A01787292

Paulina Cortez Balvanera | A01782041

María Espínola Forcén | A01787172

Integración de seguridad informática en redes y sistemas de software (Gpo 421)

23 de Octubre de 2026

Sección I: Inteligencia Artificial

Lista de los diferentes prompts utilizados para asegurar que todo el funcionamiento fuera correcto:

1. Prompt:

   "Debo calcular peso y talla estable para edad utilizando como base de referencia las tablas vigentes de la CDC, se evalúen los parámetros ideales para determinar su estado nutricio, generando las gráficas correspondientes de"Peso para la Talla", "Talla para la Edad" e "IMC", en sus diferentes categorías. Este es el archivo donde despliego esos resultados, como se puede entregar la gráfica y cálculos correspondientes sin cambiar el formato y estructura de la página:"

   Respuesta:

   Tu página ahora calcula z-score, percentil y categoría para IMC para la edad, talla para la edad y peso para la talla, y dibuja las tres gráficas. No pude ejecutarlo ni descargar las tablas CDC desde aquí. Lo escribí con cuidado, pero no está probado, y las tablas tienes que agregarlas tú.

Archivos

- PesoYTalla.tsx: es tu mismo archivo, con los mismos estilos y estructura. Solo cambió el cálculo y se agregaron las líneas de resultado y las gráficas. Se muestran después del cálculo, debajo del Alert, y los botones siguen igual.
- cdc.ts: lee las tablas, interpola y calcula z y percentil. Los puntos de corte de las categorías están en tres funciones al inicio, por si quieres ajustarlos.
- GraficaCrecimiento.tsx: dibuja las curvas de percentiles en SVG con el punto del paciente. No agrega dependencias y sigue el tema claro/oscuro.

Categorías que aplica

- **IMC para la edad:** bajo peso (<P5), saludable (P5 a <P85), sobrepeso (P85 a <P95) y obesidad (≥P95). La obesidad se divide en clases 1, 2 y 3 según qué tanto pasa del P95.
- **Talla para la edad:** talla baja (<P3), normal (P3–P97) y alta (>P97).
- **Peso para la talla:** bajo peso (<P5), adecuado (P5 a <P85), riesgo de sobrepeso (P85 a <P95) y sobrepeso (≥P95).

2. Prompt:

   "Este es el tsx de perimetro cefalico, solo agrega o modifica los calculos para ser lo mas preciso posible de acuerdo a cdc:"

   Respuesta:

Para que coincida con CDC, tomé la tabla oficial de percentiles de CDC (la de perímetro cefálico de 0 a 36 meses, niños y niñas) y la metí directo en el archivo. Probé las funciones de cálculo con valores de esa tabla y dieron lo esperado: la mediana sale en percentil 50 y el valor del P97 sale en 97.

Qué calcula
La edad en meses con decimales, a partir de la fecha de nacimiento (agregué ese campo a la pantalla, porque sin la edad no hay forma de ubicar al niño en la curva).
Interpola la tabla CDC a la edad exacta del niño y ubica el perímetro entre las curvas de percentil (P3 a P97).

Con eso obtiene el z-score y el percentil, y muestra la mediana de referencia para esa edad y sexo. La categoría sale del z-score: menor a −3 microcefalia severa, entre −3 y −2 microcefalia, entre −2 y +2 normal, y mayor a +2 microcefalia (severa por encima de +3).

Qué tan preciso es: dentro de las curvas P3 a P97 coincide con la tabla CDC. Fuera de ellas (por ejemplo, un percentil menor a 3) es una estimación por extrapolación, no el valor exacto del método LMS completo. Para clasificar a un niño como “microcefalia” o “normal” es suficiente, pero no esperes los mismos decimales que una calculadora CDC en los extremos.

3. Prompt:

   “Dame el css para el archivo peso y talla para que quede como el prototipo e indicame donde vincularlo en el tsx”

   Respuesta:

   Para que tu calculadora Peso y Talla quede como el prototipo que compartiste, vamos a utilizar un archivo PesoYTalla.css con el estilo rosa y verde oliva, tarjetas blancas, encabezados grises y botones personalizados.

   El diseño también será compatible con el modo claro y oscuro de Material UI.

   → *PesoYTalla.css*

   ¿Dónde vincularlo en pesoytalla.tsx?

   En la parte superior del archivo src/pesoytalla/pesoytalla.tsx, junto con los demás imports, agrega:

   ```
   import "./PesoYTalla.css";
   ```

   ¿Cómo se conectan las clases del CSS con el TSX?

   Debes utilizar className en los elementos correspondientes:

Elemento	Clase CSS
Contenedor principal	peso-page
Tarjeta blanca	peso-card
Encabezado	peso-header
Título rosa	peso-title
Línea gris bajo el título	peso-subtitle
Icono circular	peso-mascota
Barra de Instrucciones	peso-section
Etiquetas verdes	peso-label
Campos de peso y altura	peso-measure-row
Contenedor de botones	peso-actions
Botón Guardar	peso-guardar

4. Prompt:

   “Con el backend corriendo en localhost, inicia sesión con el usuario "demo.coord.norte" usando la contraseña correcta, que está en el archivo cuentas-demo.txt. Necesito saber como hacer para que se pueda leer la contraseña y guardar el token en $token”

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...Pmy9htM-rw",
  "usuario": {
    "id": "demo.coord.norte",
    "fullName": "Laura Méndez (coord. Norte)",
    "rol": "coordinador",
    "region": "Norte"
  }
}
```


5. Prompt:

"Con la contraseña como recibo el Token de JWT y lo verifico"

Funciona en dos partes: el **login** revisa la contraseña y te entrega el token, y un **middleware** revisa ese token en cada petición.

1. Login: contraseña → token

El usuario manda su usuario y su contraseña. Buscas el hash guardado en la base, lo comparas con Argon2 y, si coincide, firmas un JWT:

```
import express from "express";
import argon2 from "argon2";
import jwt from "jsonwebtoken";

const router = express.Router();

router.post("/login", async (req, res) => {
  const { username, password } = req.body;

  // 1. Buscar al usuario en la base
  const usuario = await db.collection("usuarios").findOne({ usuario: username });

  // 2. Comparar la contraseña con el hash guardado
  const valida = usuario && (await argon2.verify(usuario.passwordHash, password));
  if (!valida) {
    // Mismo mensaje si falla el usuario o la contraseña, para no dar pistas
    return res.status(401).json({ error: "Usuario o contraseña incorrectos" });
  }

  // 3. Firmar el token con datos básicos (nunca la contraseña)
  const token = jwt.sign(
    { sub: usuario._id.toString(), nombre: usuario.nombre, rol: usuario.rol },
    process.env.JWT_SECRET,
    { algorithm: "HS256", expiresIn: "1h" }
  );

  res.json({ token });
```

2. Verificar el token en cada petición

```
export function requiereLogin(req, res, next) {
  const encabezado = req.headers.authorization ?? "";
  const [tipo, token] = encabezado.split(" ");

  if (tipo !== "Bearer" || !token) {
    return res.status(401).json({ error: "Falta el token" });
  }

  try {
    // Revisa la firma y la fecha de expiración
    req.usuario = jwt.verify(token, process.env.JWT_SECRET, { algorithms: ["HS256"] });
    next(); // token válido → sigue a la ruta
  } catch {
    res.status(401).json({ error: "Token inválido o caducado" });
  }
}

// Todo lo que va después de esta línea pide token
app.use("/api", requiereLogin);
```

Puntos importantes

- `JWT_SECRET` va en el archivo `.env` y nunca se sube a GitHub. Quien tenga esa clave puede fabricar tokens válidos.
- `algorithms: ["HS256"]` evita el ataque del token "sin firma" (`alg: none`).
- Si alguien cambia el contenido del token, por ejemplo su rol, la firma deja de coincidir y `jwt.verify` lo rechaza.
- El token **no está cifrado, solo firmado**. Cualquiera puede leer su contenido, así que no metas datos sensibles.

6. Prompt:

"Enseñame como implemento argon2 y explicame como funciona"

¿Qué es Argon2?

Argon2 es un algoritmo de **hash para contraseñas**. Ganó la Password Hashing Competition en 2015 y hoy es el que recomienda OWASP.

Un **hash** es de un solo sentido: conviertes la contraseña en un texto raro y **no se puede regresar** a la original. Para revisar un login no "descifras" nada. Vuelves a calcular el hash con la contraseña que escribió el usuario y comparas los dos.

```
"MiClave123"  →  argon2  →  $argon2id$v=19$m=65536,t=3,p=4$c2FsdGFsZWF0b3Jp$Xk9...
                 (no hay forma de regresar)
```

¿Por qué Argon2 y no SHA-256?

SHA-256 es **demasiado rápido**: una tarjeta gráfica prueba miles de millones de contraseñas por segundo. Argon2 es **lento a propósito** y además **gasta mucha memoria**, así que atacarlo en masa sale carísimo.

Sus 3 variantes

Variante	Para qué sirve
Argon2d	Resiste ataques con GPU, pero es vulnerable a ataques de canal lateral
Argon2i	Resiste ataques de canal lateral
Argon2id	Combina las dos ventajas. **Es la recomendada**

Implementación en Node.js

2. Crear un archivo `passwords.js`

```
import argon2 from "argon2";

// Configuración (valores que recomienda OWASP)
const OPCIONES = {
  type: argon2.argon2id, // la variante recomendada
  memoryCost: 65536,     // 64 MB de memoria por cada hash
  timeCost: 3,           // 3 pasadas sobre esa memoria
  parallelism: 4,        // 4 hilos
};

// Convierte la contraseña en hash (al crear un usuario)
export async function hashPassword(password) {
  return argon2.hash(password, OPCIONES);
}

// Compara la contraseña escrita contra el hash guardado (al hacer login)
export async function verificarPassword(hash, password) {
  try {
    return await argon2.verify(hash, password);
  } catch {
    return false; // si el hash está dañado, no deja entrar
  }
}
```

3. Al crear un usuario

```
import { hashPassword } from "./passwords.js";

router.post("/users", async (req, res) => {
  const { usuario, nombre, password, rol } = req.body;

  const passwordHash = await hashPassword(password);

  await db.collection("usuarios").insertOne({
    usuario,
    nombre,
    rol,
    passwordHash, // se guarda SOLO el hash, nunca la contraseña
  });

  res.status(201).json({ usuario, nombre, rol });
});
```

4. Al iniciar sesión

```
import { verificarPassword } from "./passwords.js";

router.post("/login", async (req, res) => {
  const { username, password } = req.body;
  const usuario = await db.collection("usuarios").findOne({ usuario: username });

  const valida = usuario && (await verificarPassword(usuario.passwordHash, password));
  if (!valida) return res.status(401).json({ error: "Usuario o contraseña incorrectos" });

  // ...aquí se genera el token JWT
});
```

7. Prompt:

"Con lo que ya tengo de los roles como le hago para que no puedan ver toda la informacion dependiendo de su rol? Me explicas porfa"

Se hace en **dos capas** y siempre en el **servidor**. Esconder botones en el frontend no protege nada, porque cualquiera puede llamar al API directamente.

Capa	Pregunta que responde	Error si falla
1. Permiso por rol	¿Este rol **puede hacer** esta acción?	403
2. Filtro por registro	¿Cuáles **datos** le tocan?	Simplemente no los ve, o 404

Los ejemplos usan cuatro roles de muestra (`admin`, `coordinador`, `estancia`, `familia`). Cámbialos por los nombres y campos que tengas tú.

Capa 1: qué rol puede usar cada ruta

El rol sale del **token** (`req.usuario.rol`), no de lo que mande el usuario. Como el token está firmado, no lo puede cambiar.

```
// middleware/roles.js
export function requiereRol(...rolesPermitidos) {
  return (req, res, next) => {
    if (!rolesPermitidos.includes(req.usuario.rol)) {
      return res.status(403).json({ error: "No tienes permiso para esta acción" });
    }
    next();
  };
}
```

Se usa en cada ruta:

```
// Solo el admin administra usuarios
app.use("/api/users", requiereRol("admin"), rutasUsuarios);

// Todos pueden consultar, pero solo algunos capturan o borran
router.get("/", listar);
router.post("/", requiereRol("admin", "estancia"), crear);
router.delete("/:id", requiereRol("admin", "coordinador"), borrar);
```

Capa 2: qué datos ve cada quien

Aquí está lo importante. Aunque dos roles entren a la **misma ruta**, cada uno debe recibir **solo sus registros**. Para eso se arma un filtro de MongoDB según el rol:

```
// Devuelve el filtro que le toca a cada usuario
export function filtroAcceso(usuario) {
  switch (usuario.rol) {
    case "admin":
      return {};                                  // ve todo
    case "coordinador":
      return { region: usuario.region };          // solo su región
    case "estancia":
      return { ebdi: usuario.ebdi };              // solo su estancia
    case "familia":
      return { "paciente.usuario": usuario.sub }; // solo su hijo o hija
    default:
      return { _id: { $exists: false } };         // rol desconocido: no ve nada
  }
}
```

Para que funcione, la **región o la EBDI va dentro del token** cuando se hace el login:

```
const token = jwt.sign(
  { sub: usuario._id.toString(), rol: usuario.rol, region: usuario.region, ebdi: usuario.ebdi },
  process.env.JWT_SECRET,
  { algorithm: "HS256", expiresIn: "1h" }
);
```

Errores que hay que evitar

1. **Confiar en el frontend.** Ocultar un botón no es seguridad; el servidor siempre tiene que revisar.
2. **Tomar la región o el rol del `body` o de la URL.** Siempre van del token.
3. **Poner el filtro de acceso antes de los filtros del usuario.** Con `?region=Sur`, un coordinador del Norte podría ver otra región.
4. **Proteger la lista pero no el `/:id`.** Es el hueco IDOR del ejemplo de arriba.
5. **No tener un caso `default`.** Un rol nuevo o mal escrito no debería ver todo.

8. Prompt:

"Ya tengo mongoDB explicame como conectarlo usando express"

3. Crear el archivo `db.js` (la conexión)

```
import { MongoClient } from "mongodb";

// Cliente único para toda la app
const cliente = new MongoClient(process.env.MONGODB_URI);
let db;

// Se llama una vez al arrancar
export async function conectar() {
  await cliente.connect();
  db = cliente.db(process.env.DB_NAME);
  await db.command({ ping: 1 }); // confirma que sí responde
  console.log("✅ Conectado a MongoDB");
}

// Las rutas usan esto para obtener la base
export function getDb() {
  if (!db) throw new Error("La base todavía no está conectada");
  return db;
}

// Cerrar al apagar el servidor
export async function desconectar() {
  await cliente.close();
}
```

4. Crear el archivo `server.js`

```
import "dotenv/config";
import express from "express";
import { conectar, desconectar } from "./db.js";
import rutasMediciones from "./routes/mediciones.js";

const app = express();
app.use(express.json());

app.get("/api/salud", (req, res) => res.json({ ok: true }));
app.use("/api/mediciones", rutasMediciones);

// Primero conecta y después levanta el servidor
try {
  await conectar();
  app.listen(process.env.PORT, () =>
    console.log(`🚀 API en http://localhost:${process.env.PORT}`)
  );
} catch (err) {
  console.error("❌ No se pudo conectar a MongoDB:", err.message);
  process.exit(1);
}

// Cierra la conexión con Ctrl + C
process.on("SIGINT", async () => {
  await desconectar();
  process.exit(0);
});
```

```
node server.js
```

Abre `http://localhost:3000/api/salud` en el navegador. Debe aparecer `{"ok":true}`.

Explicación rápida

Pieza	Qué hace
`MongoClient`	Maneja la conexión. Por dentro tiene un **pool**, un grupo de conexiones que se reutilizan
`cliente.db("issste")`	Elige la base. Si no existe, Mongo la crea al guardar el primer dato
`collection("mediciones")`	Es como una "tabla". Guarda documentos en formato JSON
`getDb()`	Comparte la misma conexión con todas las rutas

Errores comunes

- `ECONNREFUSED 127.0.0.1:27017`: el servicio de MongoDB no está prendido. En Windows ábrelo en "Servicios" o corre `net start MongoDB` como administrador.
- **Abrir una conexión en cada petición**: es lento y agota las conexiones. Por eso se conecta una sola vez.
- **Meter el `req.body` completo en la base**: alguien podría agregar campos que no esperas. Guarda solo los campos que sí aceptas.
- `new ObjectId("abc")` truena con un ID mal formado. Revísalo antes con `ObjectId.isValid`.

9. Prompt:

"Me puedes explicar cómo creó las rutas de cada tipo de medición"

Haces **una sola función** que crea las rutas y la usas para cada tipo de medición. Solo cambian la colección y la validación.

```
// routes/mediciones.js
export function crearRutas(coleccion, validar) {
  const router = Router();
  const col = () => getDb().collection(coleccion);

  router.get("/", async (req, res) => {
    res.json(await col().find(filtroAcceso(req.usuario)).toArray());
  });

  router.post("/", requiereRol("admin", "estancia"), async (req, res) => {
    const datos = validar(req.body); // revisa y calcula en el servidor
    const r = await col().insertOne({ ...datos, fecha: new Date() });
    res.status(201).json({ id: r.insertedId, ...datos });
  });

  router.delete("/:id", requiereRol("admin"), async (req, res) => {
    await col().deleteOne({ _id: new ObjectId(req.params.id), ...filtroAcceso(req.usuario) });
    res.json({ ok: true });
  });

  return router;
}
```

```
// server.js
app.use("/api/pesoytalla", crearRutas("pesoytalla", validarPesoYTalla));
app.use("/api/tallablanca", crearRutas("tallablanca", validarTalla));
app.use("/api/perimetrocefalico", crearRutas("perimetrocefalico", validarPerimetro));
```

Así escribes el código **una vez** y te sirve para los tres tipos. Si agregas otro, solo es una línea más.

10. Prompt:

"Me dieron un padron en SQL cómo lo cambio a mongoDB?"



Tienes dos caminos:

Opción 1: exportar a CSV e importar. Es la más fácil si tienes MySQL instalado.

```
SELECT * FROM padron INTO OUTFILE 'padron.csv'
FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n';
```

```
mongoimport --db issste --collection infantes --type csv --headerline --file padron.csv
```

Con `--headerline`, la primera fila del CSV se usa como nombres de los campos. Si tu CSV no trae encabezados, agrégalos o usa `--fields`.

Opción 2: un script en Node. Úsalo si quieres limpiar o reacomodar los datos.

```
import mysql from "mysql2/promise";
import { MongoClient } from "mongodb";

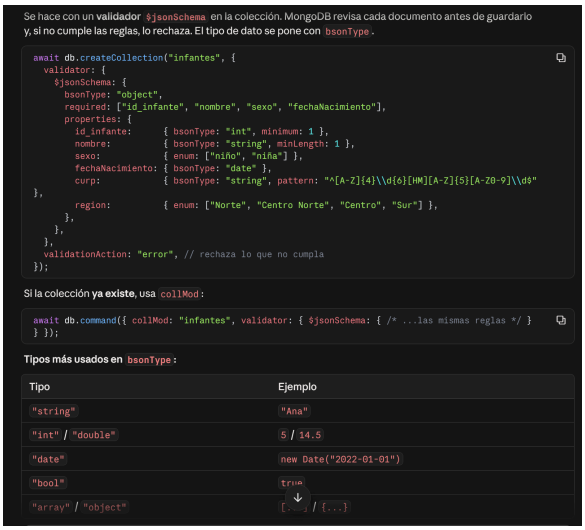
const sql = await mysql.createConnection({ host: "localhost", user: "root", database: "padron"
});
const [filas] = await sql.query("SELECT * FROM padron");

const mongo = await new MongoClient("mongodb://127.0.0.1:27017").connect();
const infantes = filas.map((f) => ({
  id_infante: f.id_infante,
  nombre: f.nombre,
  curp: f.curp,
  fechaNacimiento: new Date(f.fecha_nac),
}));

await mongo.db("issste").collection("infantes").insertMany(infantes);
console.log(`✅ ${infantes.length} registros importados`);
```

11. Prompt:

"Como puedo agregar reglas para que mongoDB revise antes de guardar a un niño? Y como pongo el tipo de dato en mongoDB?"



Se hace con un **validador** `$jsonSchema` en la colección. MongoDB revisa cada documento antes de guardarlo y, si no cumple las reglas, lo rechaza. El tipo de dato se pone con `bsonType`.

```
await db.createCollection("infantes", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["id_infante", "nombre", "sexo", "fechaNacimiento"],
      properties: {
        id_infante:      { bsonType: "int", minimum: 1 },
        nombre:          { bsonType: "string", minLength: 1 },
        sexo:            { enum: ["niño", "niña"] },
        fechaNacimiento: { bsonType: "date" },
        curp:            { bsonType: "string", pattern: "^[A-Z]{4}\\d{6}[HM][A-Z]{5}[A-Z0-9]\\d$"
},
        region:          { enum: ["Norte", "Centro Norte", "Centro", "Sur"] },
      },
    },
  },
  validationAction: "error", // rechaza lo que no cumpla
});
```

Si la colección **ya existe**, usa `collMod`:

```
await db.command({ collMod: "infantes", validator: { $jsonSchema: { /* ...las mismas reglas */ }
} });
```

Tipos más usados en `bsonType`:

Tipo	Ejemplo
`"string"`	`"Ana"`
`"int"` / `"double"`	`5` / `14.5`
`"date"`	`new Date("2022-01-01")`
`"bool"`	`true`
`"array"` / `"object"`	`[...]` / `{...}`

12. Prompt:

"Estoy teniendo un merge conflict cómo lo puedo arreglar sin quitar nada?"



Un conflicto pasa cuando tú y otra persona cambiaron **las mismas líneas** de un archivo. Git no sabe cuál versión dejar y te pide que decidas.

1. Revisa qué archivos chocaron

```
git status
```

Aparecen como `both modified`.

2. Abre cada archivo. Vas a ver algo así:

```
<<<<<<< HEAD
tu versión
=======
la versión de tu compañero
>>>>>>> origin/main
```

3. Junta las dos versiones. Deja lo de los dos y **borra las tres marcas** (`<<<<<<<`, `=======`, `>>>>>>>`):

```
tu versión
la versión de tu compañero
```

En VS Code salen botones arriba del conflicto. Para no perder nada, usa **"Accept Both Changes"** y después revisa que el código tenga sentido.

4. Marca el conflicto como resuelto y cierra el merge

```
git add .
git commit
```

Fuentes

1. Box Factura • Tu caja de facturas. (s. f.). Box Factura. https://www.boxfactura.com/calculadora/curp
2. Bro Code. (2023, 14 abril). Learn MongoDB in 1 hour 🍃 [Vídeo]. YouTube. https://www.youtube.com/watch?v=c2M-rlkkT5o
3. Caleb, J. (2023, 13 septiembre). Understanding the .env file: A Beginner's Guide to Configuration Management. John's Blog. https://johncaleb.hashnode.dev/understanding-the-env-file
4. CardContent | React Kanban Documentation. (s. f.). https://docs.svar.dev/react/kanban/api/properties/card-content/
5. CardContent API - Material UI. (s. f.). https://mui.com/material-ui/api/card-content/
6. Centro Nacional de Estadísticas de Salud & Centro Nacional para la Prevención de Enfermedades Crónicas y Promoción de la Salud. (2000). Nacimiento a 36 meses: Niños percentiles de circunferencia de la cabeza por edad y peso por estatura. En Cdc. Recuperado 23 de octubre de 2026, de https://www.cdc.gov/growthcharts/data/spanishpdf95/co06l019.pdf
7. Chaffotte, M. (2023, 12 octubre). React Admin v4 Advanced Recipes: Creating a Record Related to the Current One. Marmelab. https://marmelab.com/blog/2023/10/12/react-admin-v4-advanced-recipes-creating-a-record-related-to-the-current-one.html
8. cors middleware · Express.js. (s. f.). Express.js. https://expressjs.com/en/resources/middleware/cors/
9. Date - JavaScript | MDN. (2026, 29 mayo). https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Date
10. Environment Variables | Node.js v26.12.1 documentation. (s. f.). https://nodejs.org/api/environment_variables.html
11. freeCodeCamp. (2023, 11 abril). How to Validate Forms in React – A Step-By-Step Tutorial for Beginners. freeCodeCamp.org. https://www.freecodecamp.org/news/how-to-validate-forms-in-react/
12. Garfield, L. (2026, 29 septiembre). What is .env? A guide to understanding the .env file | Upsun. Upsun. https://upsun.com/blog/what-is-env-file/
13. GeeksforGeeks. (2021, 8 octubre). Node.js Securing Apps with Helmet.js. GeeksforGeeks. https://www.geeksforgeeks.org/node-js/node-js-securing-apps-with-helmet-js/
14. GeeksforGeeks. (2022b, septiembre 26). React MUI CardContent API. GeeksforGeeks. https://www.geeksforgeeks.org/reactjs/react-mui-cardcontent-api/
15. GeeksforGeeks. (2024, 12 junio). Use of CORS in Node.js. GeeksforGeeks. https://www.geeksforgeeks.org/node-js/use-of-cors-in-node-js/
16. GeeksforGeeks. (2025, 6 octubre). ReactJS useNavigate() hook. GeeksforGeeks. https://www.geeksforgeeks.org/reactjs/reactjs-usenavigate-hook/
17. GeeksforGeeks. (2025a, julio 23). How to use Paper Component in ReactJS ? GeeksforGeeks. https://www.geeksforgeeks.org/reactjs/how-to-use-paper-component-in-reactjs/

18. Helmet.js. (s. f.). https://helmet.js.org/
19. Introducción a Express/Node - Aprende desarrollo web | MDN. (2026, 22 septiembre). https://developer.mozilla.org/es/docs/Learn_web_development/Extensions/Server-side/Express_Nodejs/Introduction
20. Jaiswal, A. (2025, 2 febrero). A Beginner's Guide to Helmet.js: Protect Your Node.js Apps. DEV Community. https://dev.to/abhishekjaiswal_4896/a-beginners-guide-to-helmetjs-protect-your-nodejs-apps-4p2c
21. JavaScript dates. (s. f.). W3Schools. https://www.w3schools.com/js/js_dates.asp
22. Leonardo Jose. (2024, 13 marzo). Node + ExpressJS. Primeros pasos. [Vídeo]. YouTube. https://www.youtube.com/watch?v=Nsc02pc_xVc
23. MongoDB. (2025, 20 octubre). MySQL to MongoDB Migration Guide. MongoDB. https://www.mongodb.com/resources/solutions/use-cases/mysql-to-mongodb?locale_fallback=es
24. MongoDB. (s. f.-a). Beginners Guide: MongoDB Basics. https://www.mongodb.com/es/resources/products/fundamentals/basics
25. MongoDB. (s. f.). MongoDB Atlas: Cloud Document Database. https://www.mongodb.com/es/lp/cloud/atlas/try4-reg?utm_source=google&utm_campaign=search_gs_pl_evergreen_atlas_core_prosp-brand_gic-null_emea-es_ps-all_desktop_es-es_lead&utm_term=mongo&utm_medium=cpc_paid_search&utm_ad=p&utm_ad_campaign_id=20378068766&adgroup=154980291241&cq_cmp=20378068766&gad_source=1&gad_campaignid=20378068766&gbraid=0AAAADQ1400BWmYNw-YpDLWeGJ1rMMwrS&gclid=EAIaIQobChMIhNLroYjRlwMVkJbuAR1m_jnhEAAYASAAEgLlz_D_BwE
26. National Center for Health Statistics & National Center for Chronic Disease Prevention and Health Promotion. (2000). Birth to 36 months: girls length-for-age and weight-for-age percentiles. En CDC. Recuperado 23 de octubre de 2026, de https://www.cdc.gov/growthcharts/data/set1clinical/cj41c018.pdf
27. Node.js express.js. (s. f.). W3Schools. https://www.w3schools.com/nodejs/nodejs_express.asp
28. Production best practices: security · Express.js. (s. f.). Express.js. https://expressjs.com/en/advanced/best-practice-security/
29. React Card component - Material UI. (s. f.-b). https://mui.com/material-ui/react-card/
30. React Chip component - Material UI. (s. f.-b). https://mui.com/material-ui/react-chip/
31. React DataGrid - Data Columns | REACT Documentation v26.1. (s. f.). https://js.devexpress.com/React/Documentation/Guide/UI_Components/DataGrid/Columns/Column_Types/Data_Columns/
32. React Paper component - Material UI. (s. f.). https://mui.com/material-ui/react-paper/
33. React Stack component - Material UI. (s. f.). https://mui.com/material-ui/react-stack/
34. React-Admin - Form validation. (s. f.). https://marmelab.com/react-admin/Validation.html
35. React-Admin - UseCreate. (s. f.). https://marmelab.com/react-admin/useCreate.html
36. React-Admin - UseNotify. (s. f.). https://marmelab.com/react-admin/useNotify.html
37. React-Admin - UsePermissions. (s. f.-b). https://marmelab.com/react-admin/usePermissions.html

38. React-Admin - UsePermissions. (s. f.-c). https://marmelab.com/react-admin/usePermissions.html
39. React-Admin - UseRecordContext. (s. f.-b). https://marmelab.com/react-admin/useRecordContext.html
40. React-Admin - UseRecordContext. (s. f.). https://marmelab.com/react-admin/useRecordContext.html
41. React-Admin - UseRefresh. (s. f.). https://marmelab.com/react-admin/useRefresh.html
42. React-Admin - UseTranslate. (s. f.-b). https://marmelab.com/react-admin/useTranslate.html
43. react-admin/packages/ra-core/src/controller/record/useRecordContext.ts at master · marmelab/react-admin. (s. f.). GitHub. https://github.com/marmelab/react-admin/blob/master/packages/ra-core/src/controller/record/useRecordContext.ts
44. react-native-app-helpers/react-native/hooks/useRefresh/readme.md at master · jameswilddev/react-native-app-helpers. (s. f.). GitHub. https://github.com/jameswilddev/react-native-app-helpers/blob/master/react-native/hooks/useRefresh/readme.md
45. refine.dev. (s. f.-b). usePermissions Hook | Options, Patterns & Edge Cases in Refine v5 | Refine. https://refine.dev/core/docs/authentication/hooks/use-permissions/
46. refine.dev. (s. f.-c). usePermissions Hook | Options, Patterns & Edge Cases in Refine v5 | Refine. https://refine.dev/core/docs/authentication/hooks/use-permissions/
47. refine.dev. (s. f.). useCreate Hook | Best Practices for Usage & Patterns in Refine v5 | Refine. https://refine.dev/core/docs/data/hooks/use-create/
48. Salgado, L. G. (2024, 8 noviembre). Hook UseLocation de react Router | KeepCoding Bootcamps. KeepCoding Bootcamps. https://keepcoding.io/blog/hook-uselocation-de-react-router/
49. Sofela, O. (2026, 1 octubre). The Node.js and Express.js Handbook for Beginners – Servers, Routes, Routers, and Views Explained. freeCodeCamp.org. https://www.freecodecamp.org/news/nodejs-and-expressjs-handbook-for-beginners/
50. Theming - material UI. (s. f.). https://mui.com/material-ui/customization/theming/
51. Toggle Button React component - Material-UI [v4]. (s. f.). https://v4.mui.com/es/components/toggle-button/
52. ToggleButtonGroup. (s. f.). React Aria. https://react-aria.adobe.com/ToggleButtonGroup
53. Tua Saúde. (2026, 6 octubre). ¿Cuánto medirá mi hijo?: calculadora de estatura. https://www.tuasaude.com/es/altura-estimada/
54. UseLocation. (s. f.). React Router. https://reactrouter.com/api/hooks/useLocation
55. UseNavigate | Remix. (s. f.). https://v2.remix.run/docs/hooks/use-navigate
56. UseNavigate. (s. f.). React Router. https://reactrouter.com/api/hooks/useNavigate
57. UseNotifications - Toolpad Core. (s. f.). https://mui.com/toolpad/core/react-use-notifications/

58. UseState – React. (s. f.). https://react.dev/reference/react/useState
59. UseTheme | React Navigation. (s. f.). https://reactnavigation.org/docs/use-theme/
60. Using the State Hook – React. (s. f.). React. https://legacy.reactjs.org/docs/hooks-state.html
61. Vida MRR - Programacion web. (2020, 16 enero). 40. ¿Qué es CORS y cómo configurarlo? | Curso de Node js para principiantes [Vídeo]. YouTube. https://www.youtube.com/watch?v=ElcuQfA6aNY
62. Viera, U. (s. f.). Qué es CORS y cómo usarlo en Node.js. urianviera.com. https://urianviera.com/nodejs/que-son-las-cors-en-nodejs
63. Zanini, A. (2024, 4 junio). Using Helmet in Node.js to secure your application - LogRocket Blog. LogRocket Blog. https://blog.logrocket.com/using-helmet-node-js-secure-application/